Practical course

Systems Programming

https://github.com/ls1-sys-prog-course/docs

Introduction

Systems Research Group

https://dse.in.tum.de/

About us

Systems Research Group

Our research topics include

- Operating Systems and Virtualization
- Distributed Systems and Cloud Computing
- Hardware Security and HW/OS Co-Design
- Binary Translation and Memory Models
- Quantum Software Systems
- ….

- Looking for a Master/Bachelor thesis topic? https://dse.in.tum.de/

Goals of the course

- Acquire fundamental knowledge to build robust systems
- Familiarize yourself with end-to-end system design
- Learn techniques for profiling, debugging and optimization of low-level code
- Get a good understanding of memory- and resource management
- Improve hands-on experience through a variety of programming tasks

Importantly, have fun!

Course format

- Programming assignments using Classroom 50 (on GitHub)
  - Programming exercises released almost weekly
  - Deadline of 2 - 3 weeks depending on the difficulty/workload
  - Online submission & automatic grading
- Weekly Q&A meeting (attendance is optional)
  - Question and answer session to explain and discuss each assignment
- Zulip channel for questions and discussion
  - https://zulip.cit.tum.de/#narrow/channel/3724-SysProg---General
- Grading
  - **Programming assignments (100%)** with public & private unit tests
  - No further exam / quiz / projects
- Latest Information: https://github.com/ls1-sys-prog-course/docs

About the assignment setup

- Languages
  - Choice between **C, C++ and Rust**
  - Can be switched for each task
  - Limited choice of allowed libraries (different per language)

- OS Environment information
  - All executables must run on **Linux, x86_64**
  - Use virtual machines if you run a different OS (i.e. Hyper-V on Windows)

Outline

- ~~Course introduction~~
- How to use Classroom 50
- Assignment (Task 0)

GitHub and Classroom 50

- Create GitHub account: https://github.com/
- NOTE: Students can get many benefits from GitHub
  - https://education.github.com/students
- We distribute and grade the assignments with **Classroom 50**
  - https://classroom50.org (sign in with your GitHub account)

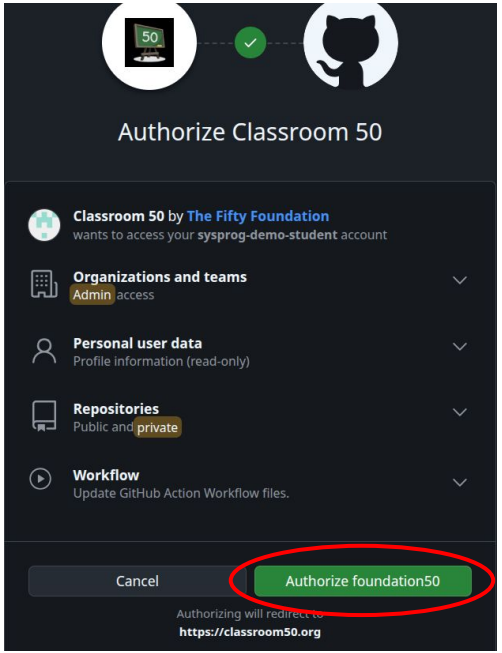
Join assignments

- First, accept **the GitHub invitation** to join the ls1-sys-prog-course organization
  - Sent by email, valid for 7 days
- Then sign in at https://classroom50.org with your GitHub account
- For each assignment, we send **an accept link** via email and Zulip
  - Example:

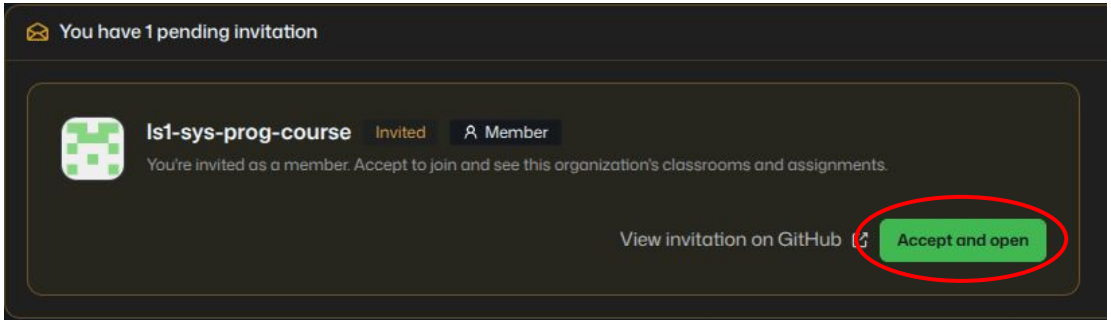
    ```
    https://classroom50.org/ls1-sys-prog-course/systems-programming-wise-2026/assignments/task0-sort/accept
    ```
- Join an assignment by clicking the link

Sign in to Classroom 50 (only first time)

- Sign in with your GitHub account and **authorize Classroom 50**
- **Accept the invitation** to the ls1-sys-prog-course organization



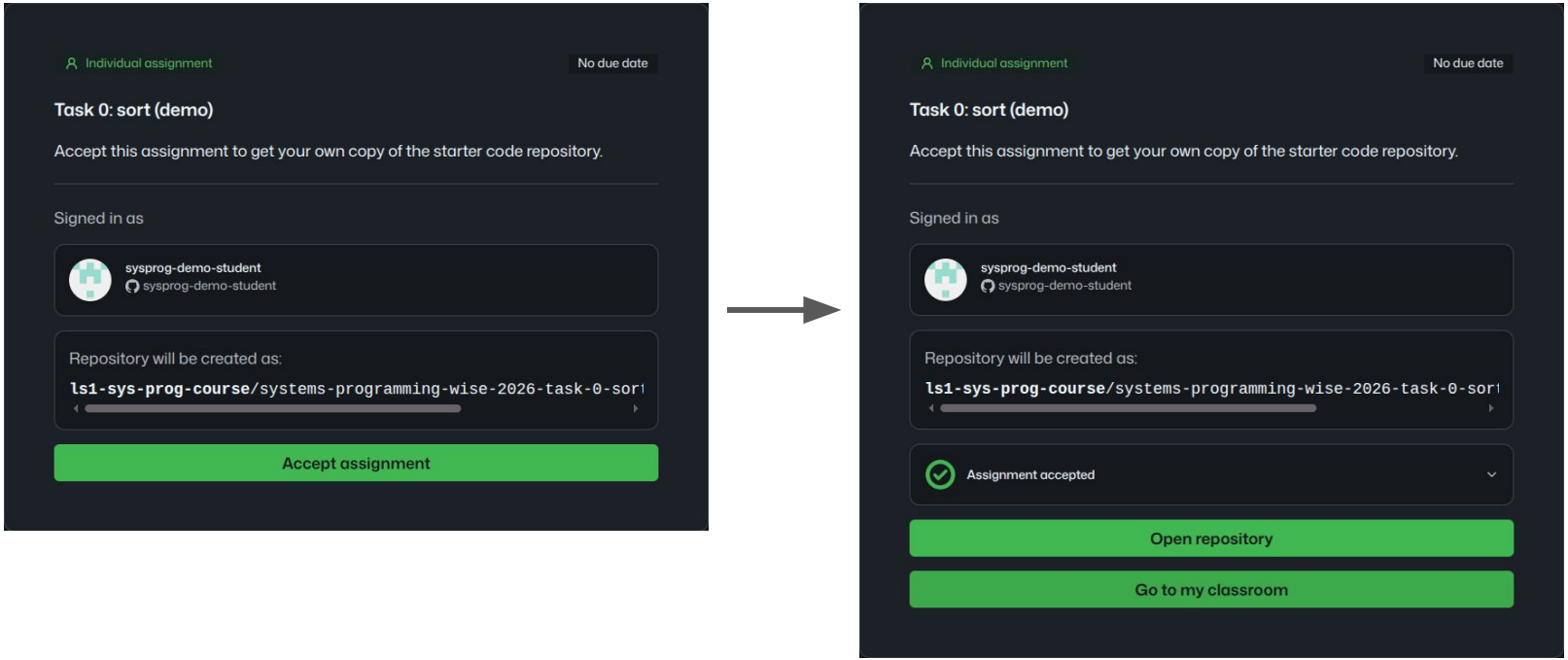
Your invitation also appears on the Classroom 50 home page:



Click "Accept and open"

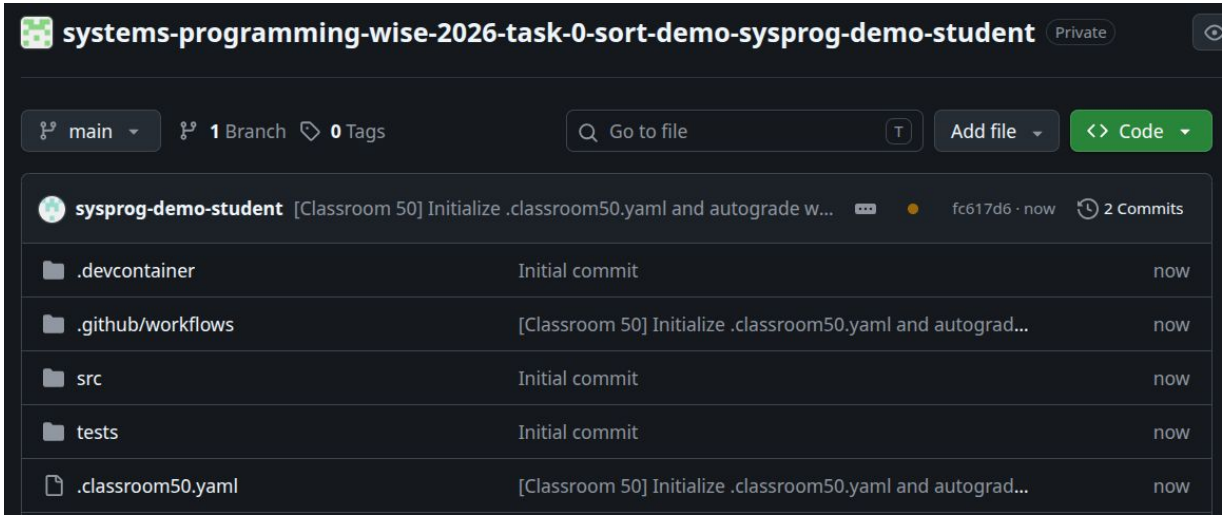
Accept the assignment

- Open the accept link, check your account and click **Accept assignment**
- When the setup is complete, click **Open repository**



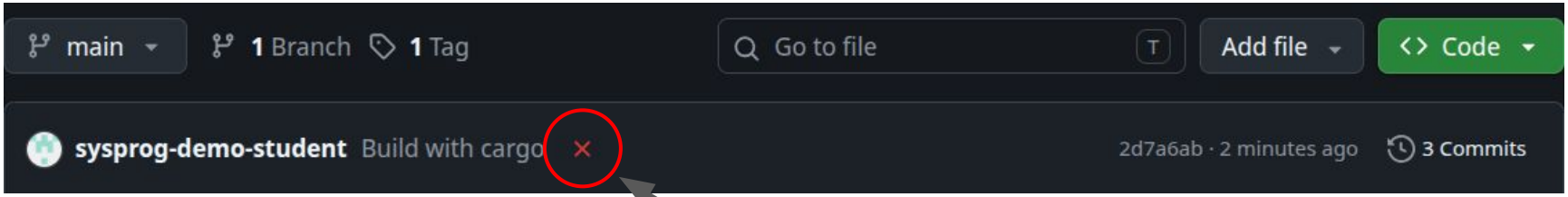
Let’s do the task!

- Your repository for the task is automatically created in ls1-sys-prog-course organization
- Push your code (solution) to this repository
- **Do not edit tests, .github and .classroom50.yaml** (we check their integrity)



Check your score and test results

- **Every push to the main branch is automatically tested** by GitHub CI
- A red cross or a green check mark appears next to your commit



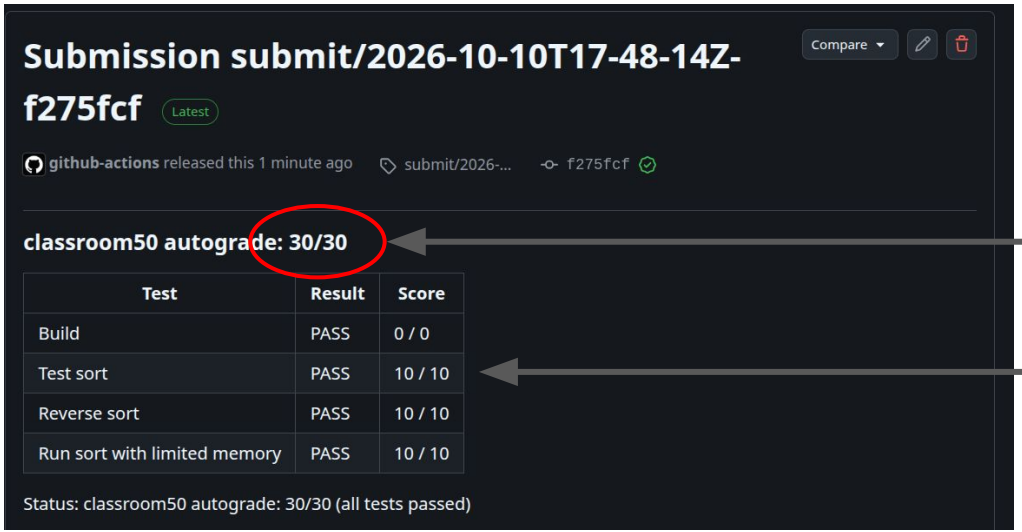
Some tests failed
Click the mark to check the detail



All tests passed

Check your score and test results (contd.)

- Your score is published on the **Releases** page of your repository
- NOTE: the run in the Actions tab is green even when tests fail
  - Check the mark next to your commit, or the release

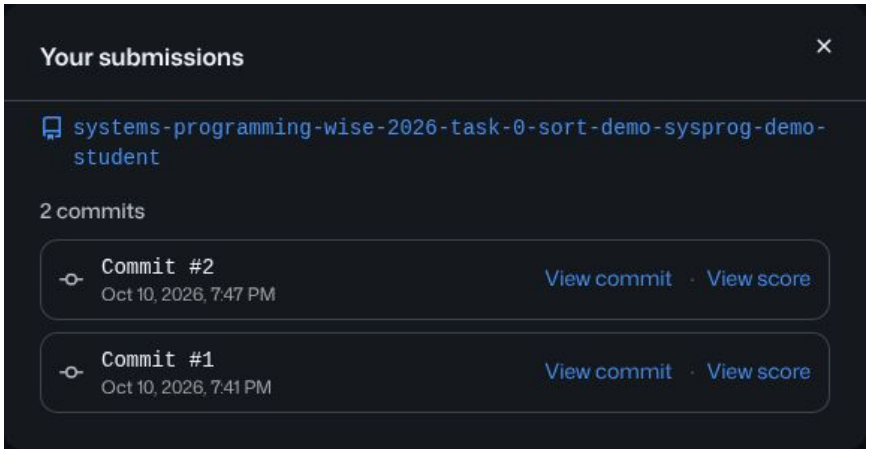


You can find your score here

Find the result of each test here

NOTE

- Your score will be the result of **the latest commit before the deadline**
- Only commits on the main branch count
- Every push to main is a submission: see them on classroom50.org (**My submission**)



Outline

- ~~Course introduction~~
- ~~How to use Classroom 50~~
- Assignment (Task 0)

Assignment (task 0)

- Implement sort (score: 0 points)
- Goal: get used to GitHub and Classroom 50
- Refer to the task repository for the detail

How to find documentation

- References/documentation of your language:
  - C: https://en.cppreference.com/w/c
  - C++: https://en.cppreference.com/w/
  - Rust standard library: https://doc.rust-lang.org/std/index.html
- System call / Operating system documentation:
  - Each system call has a different page in manpage chapter **2**
  - On command line:
    - `$ man 2 read`
  - Online:
    - https://man7.org/linux/man-pages/man2/read.2.html

How to find examples

- Documentation is often a lie
  - Implementation sometimes easier to gasp than description
- Learn to Read the Source, Luke!
  - https://github.com/systemd/systemd (implements every syscall; modern C codebase)
  - Read-able libc: https://musl.libc.org/
  - Linux kernel: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/
    - Or search online: https://elixir.bootlin.com/linux/latest/source
- Get familiar with a code search tool:
  - Example: ripgrep: https://github.com/BurntSushi/ripgrep
  - Online search (github code search, elixir, etc.)

When the code does not work...

- Use a debugger: gdb, rust-gdb
  - Learn most common commands (break; next; continue; print)
  - Enable debug symbols: `$ cc –Og –g main.c -o main`
  - Nicer graphical Interface: https://www.gdbgui.com/
  - Advanced (text) interface for low-level debugging: https://github.com/hugsy/gef
- Printf-Debugging:
  - Useful when debugging parallel issues/distributed code
  - C: fprintf(stderr, "%s() at %s:%d: some var: %d\n", __func__, __FILE__, __LINE__, some_var);
  - dbg! Macro in rust: https://doc.rust-lang.org/edition-guide/rust-next/dbg-macro.html

Latest information

https://github.com/ls1-sys-prog-course/docs